# CityMart E-Commerce Platform

Design & Deployment of a Production-Grade 3-Tier Architecture on AWS

Akosa Samuel Onyejekwe

Programme / Academy: Cloud Computing/ Cloudboosta

April 2026

# Problem Statement

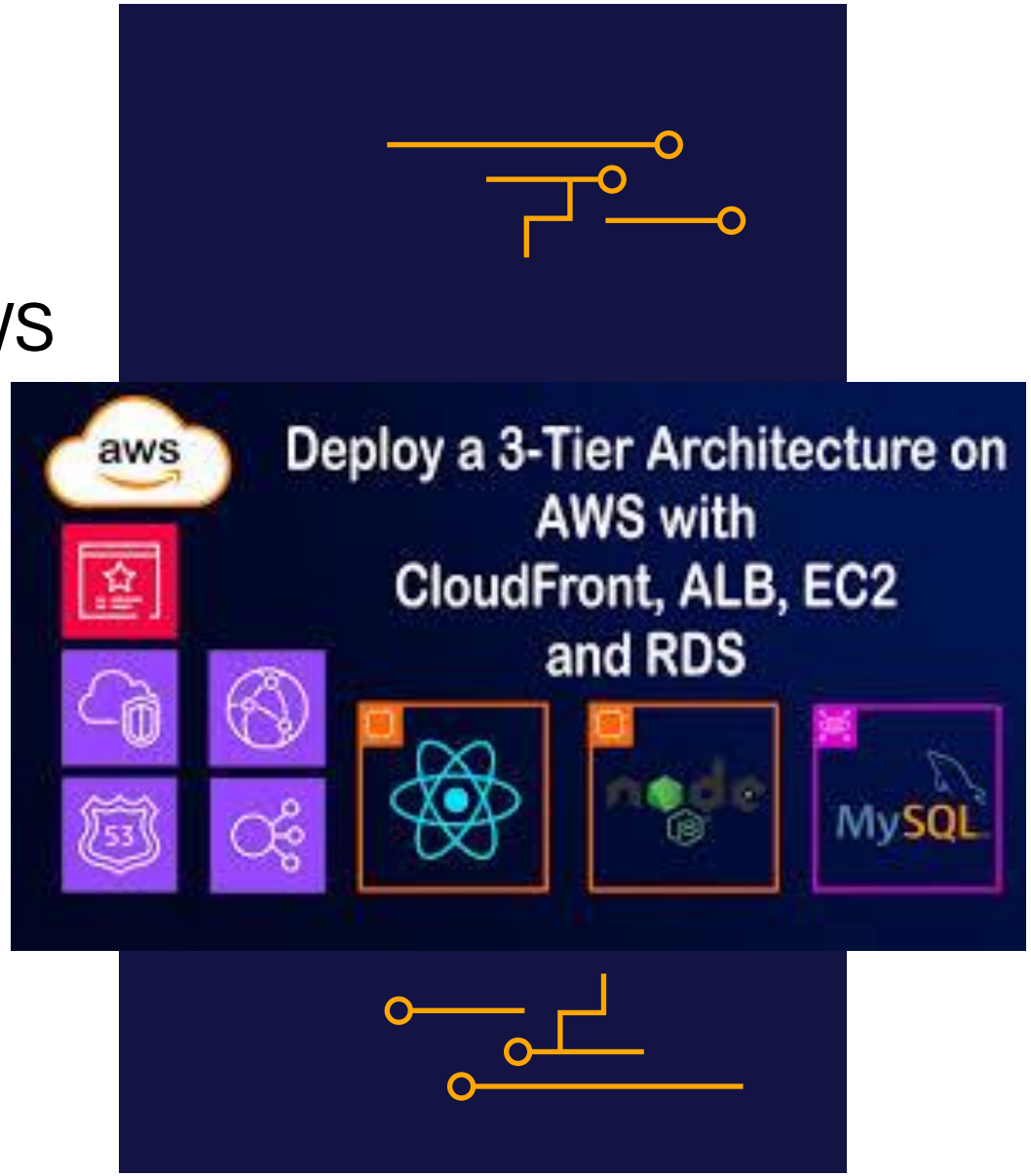
- Traditional retail systems lack:
  - Scalability
  - High availability
  - Secure remote access
- Need for:
  - Cloud-native digital platform
  - Reliable order processing system

# Project Objectives

- Enable digital customer interaction
- Ensure high availability
- Achieve scalability & elasticity
- Enforce strong security
- Build production-ready architecture

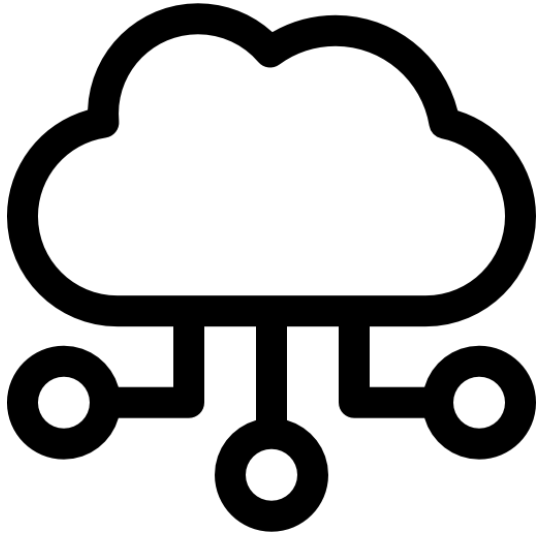
# Architecture Overview

- Cloud-native 3-tier architecture on AWS
- Components:
  - CloudFront (CDN)
  - Route 53 (DNS)
  - ALB (Load balancing)
  - EC2 (Compute)
  - RDS (Database)
  - S3 (Deployment storage)



# 3-Tier-Architecture diagram

- https://drive.google.com/file/d/19X9crdE447n77kD_vR6bTXA9Bj20-6Db/view?usp=drive_link



# End-to-End Request Flow

- **User Request Path:**
- User → Route53 → CloudFront → Public ALB → Frontend EC2 → Internal ALB → Backend EC2 → RDS → Response

# Three-Tier Architecture Model

- Presentation Layer (Frontend)
- Application Layer (Backend)
- Data Layer (Database)

- Separation of concerns
- Independent scaling
- Security isolation

# Presentation Tier (Frontend)

- CloudFront (global edge entry)
- Public ALB
- EC2 (Nginx servers)
- Static assets (HTML, JS, CSS)

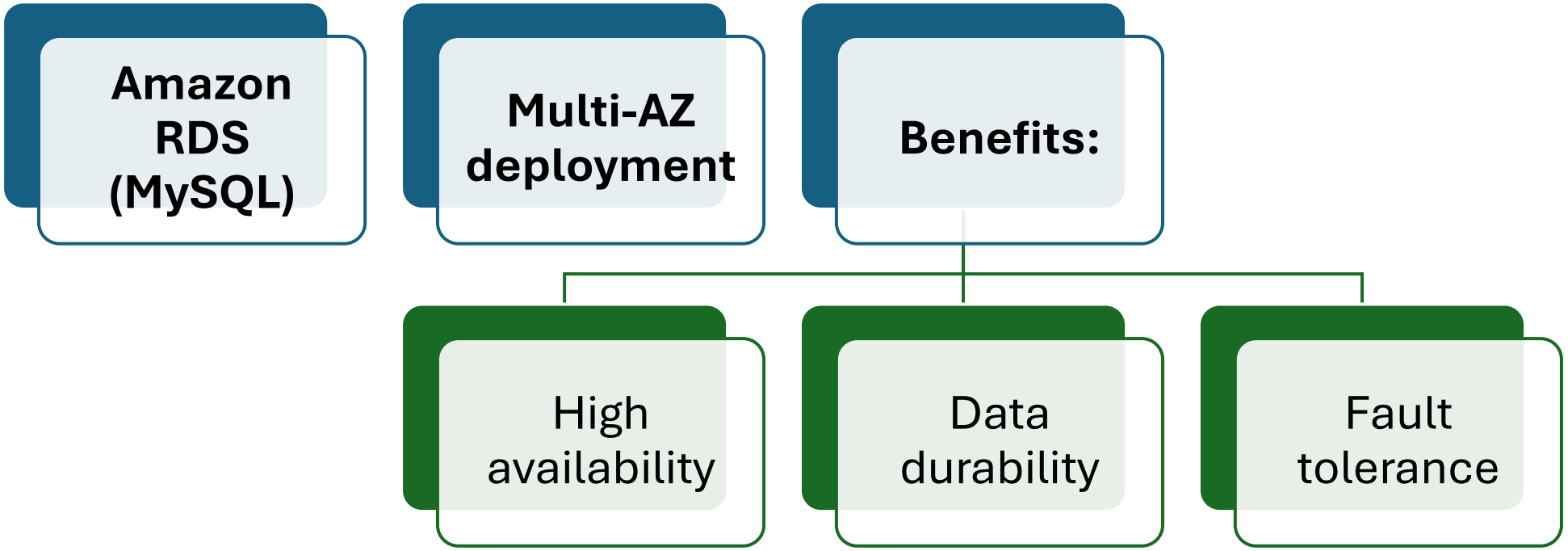
- **Functions:**
  - Serve UI
  - Cache content
  - Route API requests

# Application Tier (Backend)

- Internal ALB
- EC2 (Apache + PHP APIs)

- **Responsibilities:**
  - Business logic
  - API processing
  - Database interaction
    - Fully private (no internet access)

# Data Tier


Amazon RDS (MySQL)
Multi-AZ deployment
Benefits:
High availability
Data durability
Fault tolerance

# Supporting Services

- CloudFront → Caching & latency reduction
- Route 53 → Domain routing
- ACM → TLS/HTTPS
- S3 → Application deployment

# Network Architecture (VPC Design)

- VPC CIDR: **10.0.0.0/16**
- Multi-AZ deployment (eu-west-2a, 2b)
- DNS enabled

# Subnet Design

- **8 Subnets (2 AZs):**
  - Public (ALB + NAT)
  - Private Web (Frontend)
  - Private App (Backend)
  - Private DB (RDS)

# CityMart VPC Subnet Layout

| Tier | Subnet Name | CIDR Block | Availability Zone | Purpose / Hosted Resources |
|---|---|---|---|---|
| Public Tier | Public Subnet A | 10.0.0.0/20 | eu-west-2a | ALB + NAT Gateway |
| Public Tier | Public Subnet B | 10.0.16.0/20 | eu-west-2b | ALB + NAT Gateway |
| Web Tier (Private) | Web Subnet A | 10.0.48.0/20 | eu-west-2a | Nginx servers |
| Web Tier (Private) | Web Subnet B | 10.0.64.0/20 | eu-west-2b | Nginx servers |
| Application Tier | App Subnet A | 10.0.96.0/20 | eu-west-2a | PHP backend |
| Application Tier | App Subnet B | 10.0.112.0/20 | eu-west-2b | PHP backend |
| Database Tier | DB Subnet A | 10.0.144.0/20 | eu-west-2a | RDS |
| Database Tier | DB Subnet B | 10.0.160.0/20 | eu-west-2b | RDS |

# Routing & Connectivity

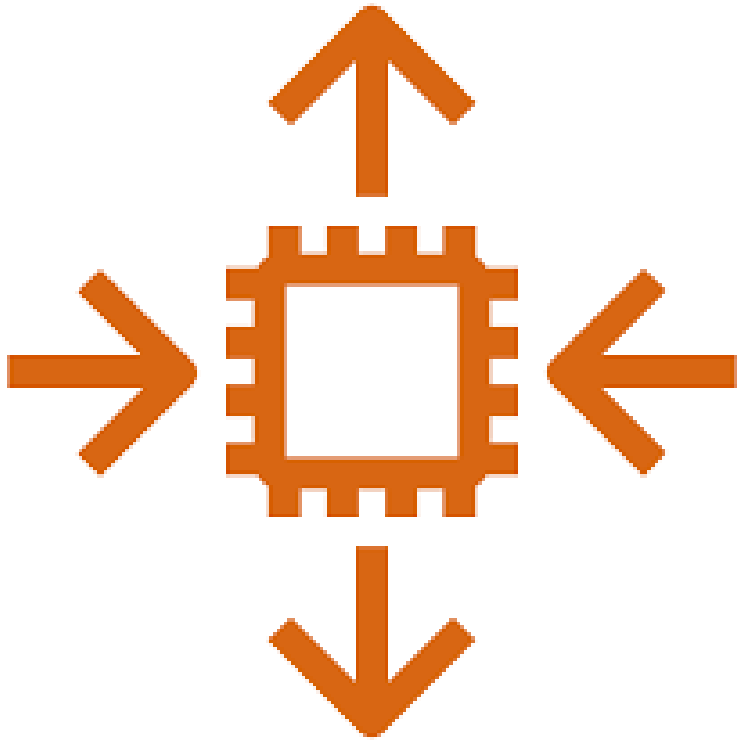
- Internet Gateway → Public access
- NAT Gateway → Private outbound access
- Route Tables:
  - Public → IGW
  - Private → NAT
- Secure traffic flow

# Security Architecture

- Layered Security Groups:
  - ALB → Frontend → Backend → DB
- Zero-trust model
- Least privilege access

# Auto Scaling & High Availability

- Auto Scaling Groups (ASG)
- Multi-AZ deployment
- Load balancing via ALB
  - Self-healing
  - Elastic scaling



# Deployment Model

- Code stored in S3
- EC2 pulls code at launch
- User data scripts automate setup
- Managed via SSM
  - Consistent deployments
  - No manual configuration

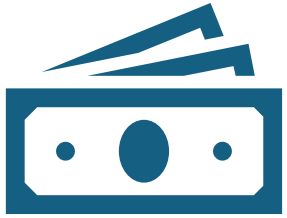
# Performance Optimisation

- CloudFront edge caching
- ALB traffic distribution
- Stateless architecture
- Low latency
- High throughput

# Cost Optimisation

Pay-as-you-go model

Auto scaling reduces idle resources



Managed services reduce ops cost

# AWS Well-Architected Alignment

Security → Layered controls

Reliability → Multi-AZ

Performance → CDN + ALB

Cost → Elastic resources

Operations → Automation

# Key Engineering Achievements

Fully segmented architecture

Secure traffic flow

Scalable infrastructure

Automated deployment

# Conclusion

Production-grade cloud architecture achieved

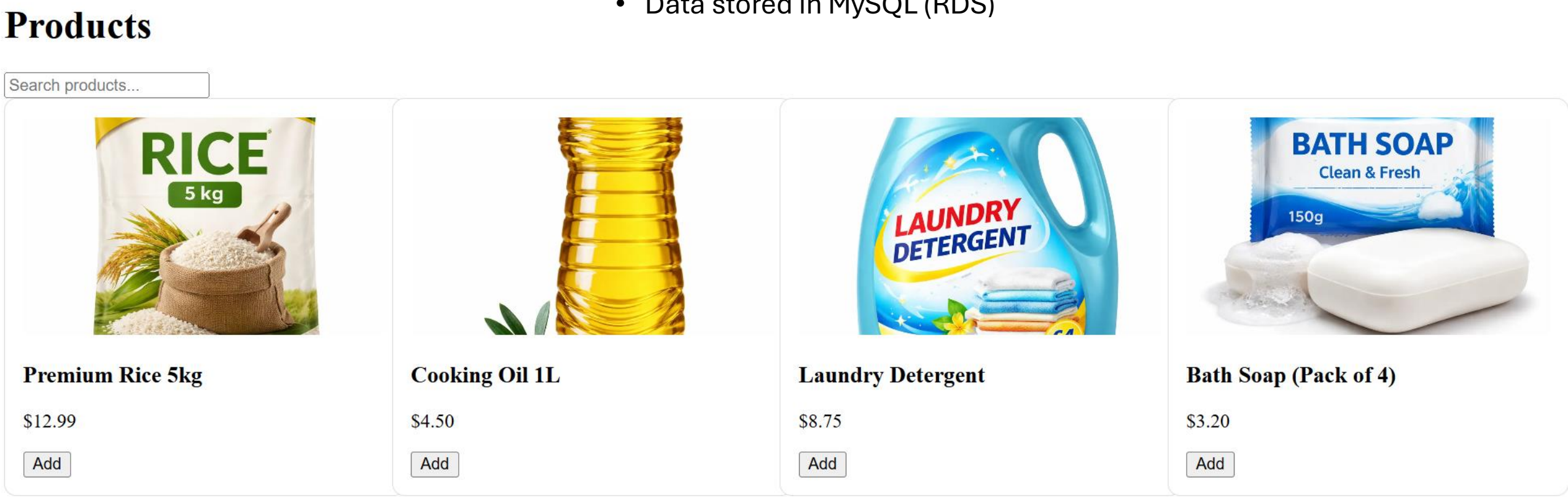
Highly scalable, secure, and fault tolerant

Ready for real-world deployment

# Functional Flow (Frontend → Backend)

**citymart.samuelonyejekwe.com**

- User browses products (Frontend – Nginx)
- Adds items to cart (client-side state)
- Submits order via API request (/api/order.php)
- Backend processes order (Apache + PHP)
- Data stored in MySQL (RDS)



# Checkout

Review your cart and submit an order request for pickup.

## Cart

Premium Rice 5kg — Qty: 1 Remove

Laundry Detergent — Qty: 2 Remove

**Total: $30.49**

## Customer details

Name

e.g., Samuel

Email

e.g., samuel@citymart.samuelonyejekwe.com

Phone

e.g., +357...

Pickup notes (optional)

e.g., pickup at 5pm

Submit Order Request

# Order request submitted ✅

Thanks! We will confirm your pickup details shortly.

Order Reference

**551212**

Continue Shopping  Home

If you did not receive a reference, please try again.

# Key Features of CityMart UI

- Simple and intuitive user interface
- End-to-end order workflow (browse → checkout → confirm)
- Dynamic interaction with backend APIs
- Lightweight and fast-loading frontend
- Designed for scalability in cloud deployment

# Thank You